



Automotive Data Transfer and Visualization Using CAN bus, Linux and Electron

Item Type	Proceedings; text
Authors	Smith, Nick; Delgado, Benni; Larsen, C.J.; Tan, Nicolas; Garcia, Gerardo; Copic, Christian
Citation	Smith, N., Delgado, B., Larsen, C., Tan, N., Garcia, G., & Copic, C. (2021). Automotive Data Transfer and Visualization Using CAN bus, Linux and Electron. International Telemetering Conference Proceedings, 56.
Publisher	International Foundation for Telemetering
Journal	International Telemetering Conference Proceedings
Rights	Copyright © held by the author; distribution rights International Foundation for Telemetering
Download date	04/06/2025 23:27:43
Item License	http://rightsstatements.org/vocab/InC/1.0/
Version	Final published version
Link to Item	http://hdl.handle.net/10150/666243

Automotive Data Transfer and Visualization Using CAN bus, Linux and Electron

International Telemetry Conference 2021

Nick Smith^{1,3} Benni Delgado² CJ Larsen²
Nicholas Tan^{3,4} Gerardo Garcia¹ Christian Copic¹

University of Arizona

April 2021

¹Electrical Engineering

²Computer Science

³Material Science

⁴Aerospace Engineering

Contents

1	Abstract	2
2	Hardware	2
3	Software	3
3.1	Motivations	3
3.2	Application Architecture	3
3.2.1	Processing	4
3.2.2	Rendering	5
3.3	Performance Improvements	8
3.3.1	Buildroot Linux	8
3.3.2	Node.js & External Dependencies	8
4	Critical Information	9
5	Complications	9
6	Future Works	9

List of Figures

1	Hardware Control Flow	2
2	Arduino Specifications	2
3	Raspberry Pi 4 Specifications	3
4	Program Control Flow	4
5	Process Module	5
6	Render Module	6
7	Dashboard GUI	7
8	Failed serial port connection	7
9	FSAE Dashboard Dependencies	8

1 Abstract

Dashboards and head-units in automobiles provide a subconscious accumulation of information to the driver, allowing for proper operation of the car. Our goal was to create a solution utilizing the effects of pre-attentive processing in the construction of our Formula SAE (Society of Automotive Engineers) race car to provide the driver with the necessary information, further advancing the performance of the car. This solution entailed a Raspberry Pi with a 3.5 inch display connected to the car's CAN Bus protocol. This protocol was configured to allow the sensors in the car, such as the throttle position sensor, lambda sensor, etc. to send information to the dashboard. With this information, we are able to decode and interpret the data into easily interpreted visualizations for the driver using a stripped down version of Linux and Electron, a tool used for building user interfaces with Node.js.

2 Hardware

Our current Engine Control Unit (ECU), is a MoTeC M150, which is a very common ECU in Formula SAE. The ECU communicates with the Raspberry Pi via CAN bus. The Pi then displays the relevant information onto the LCD display.

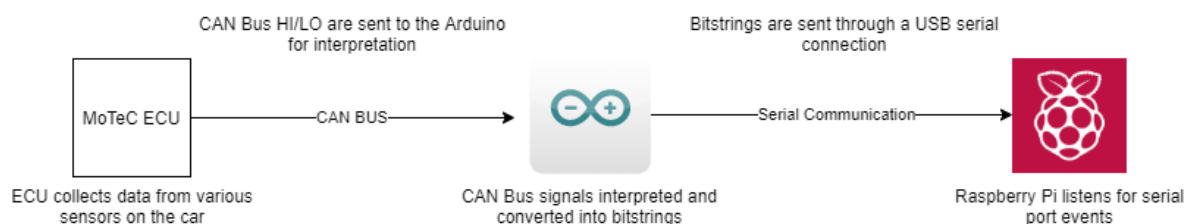


Figure 1: Hardware Control Flow

Shown above in Figure 1 is our hardware control flow. Our MoTeC ECU collects data from the onboard sensors on the car and transmits them to an Arduino using CAN Bus. The Arduino is able to interpret these signals through the use of a MCP2551 controller and decode them into bitstrings. These bitstrings are then sent through USB to the Raspberry Pi, from which they are then decoded from bitstring into human-readable integers and strings.

Microcontroller	ATmega328P
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limit)	6-20V
Digital I/O Pins	14 (of which 6 provide PWM output)
PWM Digital I/O Pins	6
Analog Input Pins	6
DC Current per I/O Pin	20 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	32 KB (ATmega328P) of which 0.5 KB used by bootloader
SRAM	2 KB (ATmega328P)
EEPROM	1 KB (ATmega328P)
Clock Speed	16 MHz
LED_BUILTIN	13
Length	68.6 mm
Width	53.4 mm
Weight	25 g

Figure 2: Arduino Specifications

Processor:	Broadcom BCM2711, quad-core Cortex-A72 (ARM v8) 64-bit SoC @ 1.5GHz
Memory:	1GB, 2GB, 4GB or 8GB LPDDR4 (depending on model) with on-die ECC
Connectivity:	2.4 GHz and 5.0 GHz IEEE 802.11b/g/n/ac wireless LAN, Bluetooth 5.0, BLE Gigabit Ethernet 2 × USB 3.0 ports 2 × USB 2.0 ports.
GPIO:	Standard 40-pin GPIO header (fully backwards-compatible with previous boards)
Video & sound:	2 × micro HDMI ports (up to 4Kp60 supported) 2-lane MIPI DSI display port 2-lane MIPI CSI camera port 4-pole stereo audio and composite video port
Multimedia:	H.265 (4Kp60 decode); H.264 (1080p60 decode, 1080p30 encode); OpenGL ES, 3.0 graphics
SD card support:	Micro SD card slot for loading operating system and data storage
Input power:	5V DC via USB-C connector (minimum 3A ¹) 5V DC via GPIO header (minimum 3A ¹) Power over Ethernet (PoE)–enabled (requires separate PoE HAT)
Environment:	Operating temperature 0–50°C
Compliance:	For a full list of local and regional product approvals, please visit https://www.raspberrypi.org/documentation/hardware/raspberrypi/conformity.md
Production lifetime:	The Raspberry Pi 4 Model B will remain in production until at least January 2026.

Figure 3: Raspberry Pi 4 Specifications

3 Software

3.1 Motivations

Our requirements for the software solution entailed that 1) the codebase must be easily maintainable and, 2) the software must be able to run on low-power hardware such as a Raspberry Pi or Arduino. This led us to utilize [Node.js](#), a server-side solution for running JavaScript, and [Electron](#), a framework for building desktop applications using a web-based stack (HTML, CSS, JavaScript). Using this stack allows us to satisfy our first requirement as a large majority of user interfaces are built off of a markup based language, thus leading to a large amount of documentation available and eliminating some of the “[Bus Factor](#)” that would occur as our team roster changes frequently. Lastly our second requirement is fulfilled by Electron, allowing for our application to run on various platforms, such as a Linux based Raspberry Pi. However, while Node.js is very fast, as it is interpreted by [Chrome’s V8 JavaScript Engine](#), a quicker solution would be to use a tool such as [QT](#) or to write the program using a lower-level language such as c++ directly. These solutions, while providing better overall performance, would end up driving the cost of operation (as [QT](#) is an enterprise solution), and increase the onboarding time for a new member to understand the codebase regarding the use of a lower-level language.

3.2 Application Architecture

At a high level, our application follows an event-driven architecture listening and responding to various different events.

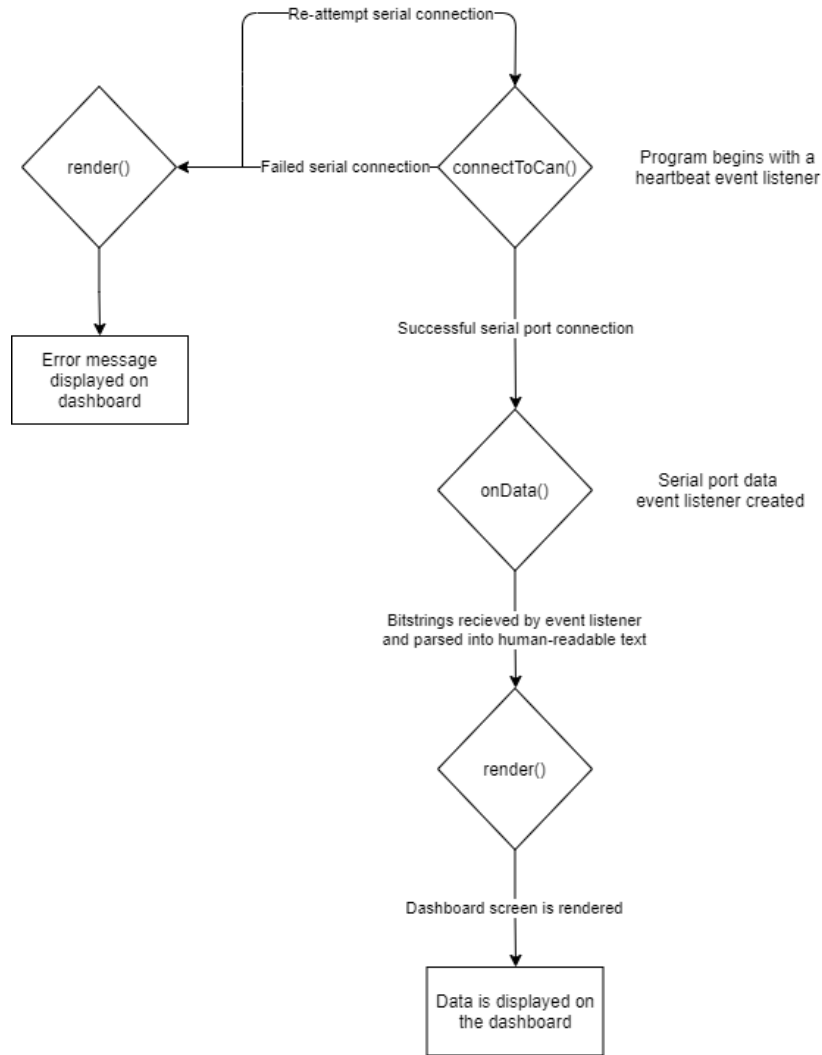


Figure 4: Program Control Flow

Again this utilizes the web-stack such that HTML/CSS controls the layout and styling of our elements and Node.js delivers our business, rendering, and input/output logic.

3.2.1 Processing

Shown in Figure 4 is a high level abstraction of the control flow our program follows. This beings with our process module, responsible for handling input/output events shown below in Figure 5.

process
+ PORT : string + BAUDRATE : int + port : SerialPort + parser : SerialPort.pipe
+ connectToCan() : void + onOpen() : void + onClose() : void + onData() : void

Figure 5: Process Module

When the application launches our main method `connectToCan()` is invoked creating a "heartbeat" event-listener. A heartbeat event, in our context, refers to any signals received from our specified serial port. Here if we receive no signal from the serial port (no heartbeat), the `render` module will be invoked with an error message notifying the driver that the dashboard is not receiving any data from the ECU. The program will then check every two seconds for a heartbeat until one is found.

Once a serial port connection is established, a `parser` object along with an `onData` event-listener is created. This event is handled by the `parser` provided by the [Node.js SerialPort library](#) responsible for processing the input signals recieved by the serial (I.E converting the electrical signals back into literal bitstrings). The bitstrings contain information regarding the current status of the car relayed from the ECU such as engine RPM, vehicle speed, and the current selected gear. Once this information has been parsed from their bitstrings to integers, they are then sent to `render` module which itself has specified functions for displaying this information to the driver.

3.2.2 Rendering

Going into another level of abstraction (Figure 6) is our render module, responsible for creating the text and visualizations for the dashboard. There are five main exported functions for rendering: `fillSpeedo()`, `fillTach()`, `fillGear`, `sendWarning()`, and `cancelWarning()`, and one private helper function `mapValue()`.

render
+ body : Element + app : Element + warning : Element + tach : Element + speedo : Element + gearElem : Element + canvas : HTMLCanvasElement + tachBar : HTMLCanvasElement
- mapValue(float[]) : float + fillTach(int) : void + fillSpeedo(int) : void + fillGear(int) : void + sendWarning(string) : void + cancelWarning() : void

Figure 6: Render Module

Fields

Shown above in Figure 6 is our module diagram showing the various fields and functions that are encapsulated by render. The fields in render correspond to DOM elements in our HTML file, for instance the field `speedo` corresponds to the HTML div containing the speedometer text. The two "exceptions" here are `canvas` and `tachBar` as they refer to an HTML canvas and the canvas' two dimensional context, used for creating the tachometer visualization.

Functions

Shown below in Figure 7 is what our LCD displays to the driver, rendered by manipulating the DOM in our five main render functions.

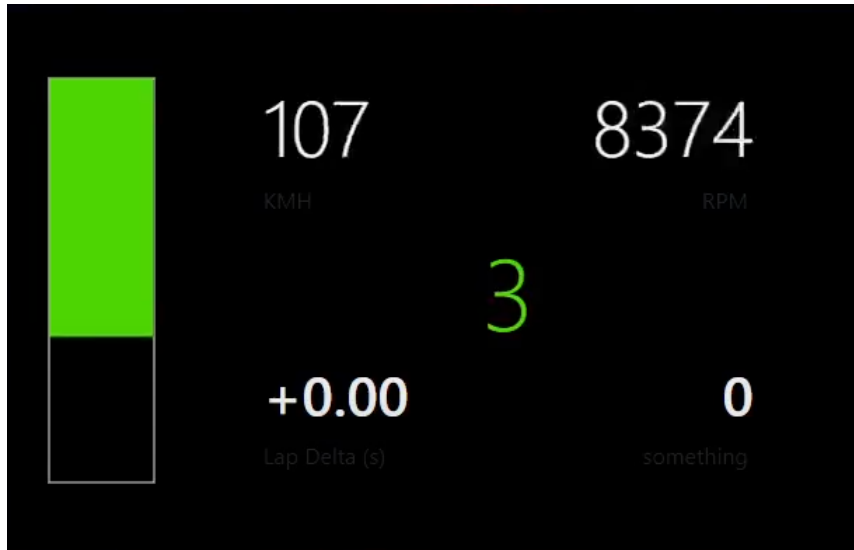


Figure 7: Dashboard GUI

Stated above, there are five main functions for rendering the dashboard content. The functions `fillSpeedo()`, and `fillGear()` are perhaps the simplest of the bunch as they change the HTML text inside their corresponding DOM elements defined by the module fields `speedo` and `gearElem` respectively. However, `fillTach()` is responsible for generating the "tachBar" visualization, as well as filling in the RPM into the tachometer text.

The "tachBar" (Figure 7 left-most component) is a large rectangle that fills and changes color when the engine speed gets to a certain limit, signaling the driver to shift gears. This is done by using the HTML canvas to create our primitive rectangle shapes, then using our helper function `mapValue()` to map literal engine RPM to a percentage of the bar that should be filled. The visualization takes advantage of pre-attentive processing, as the change in color and size of the bar subconsciously attracts the attention of the driver. Changing gears at the right time is not only imperative to our performance on the racetrack, but also protects the engine from revving either too high or low, thus this is a situation in which we actively need the drivers attention.

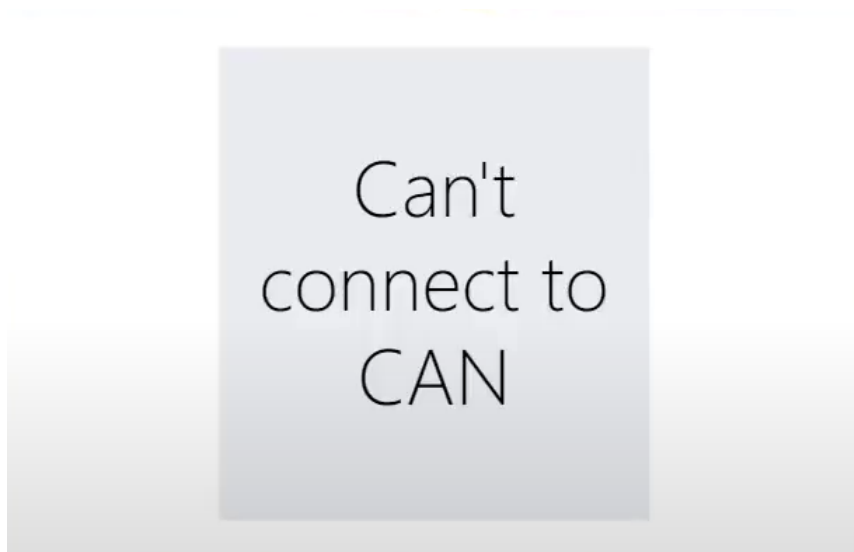


Figure 8: Failed serial port connection

Lastly, shown in Figure 8 above, is an example of the `sendWarning()` function. This function is invoked with a string representing an error message to show to the driver. This usually occurs during a critical failure such as an dead heartbeat signal, as no other information regarding the vehicle will be shown in this state. The applications will be in this 'failure' state until the function `cancelWarning()` is called reverting back to the default screen shown in Figure 7.

3.3 Performance Improvements

This system was powered by a Raspberry Pi Model 4 B, featuring two gigabytes of DDR4 memory, a class 10 SD card, and 1.5 Ghz quad-core 64 bit CPU, an extremely fast system considering our application. Yet we encountered two main areas in which we could greatly improve performance and boot time.

3.3.1 Buildroot Linux

Our largest source increasing our boot time (time taken from initial power to application launch) was the operating system itself. Our original version of this system was powered by Raspberry Pi OS (previously named Raspbian), a Debian based operating system made specifically for the Raspberry Pi. With this operating system we encountered boot times of close to 30 seconds before our application was fully running. This was considerably slower than we aimed to have as if we compared this to our road cars, we expect our headunit/radio to be up and running within a few seconds of turning the car on.

We believed this latency was due to all of the onboard processes that were set to boot before our main process was unblocked for launch. This was confirmed by an analysis using `systemd`'s 'blame' and 'analyze' programs. Specifically the processes regarding wireless LAN, bluetooth, cron jobs, splash screens, and the desktop graphical user interface were significantly slowing down our boot time. This was originally solved by disabling our wireless LAN, bluetooth, and splash screens decreasing our boot time to around 20 - 25 seconds, still a relatively long time. At this point we cannot disable any more services as the operating system, designed to be a full desktop OS, required them. This lead us to research into [Buildroot Linux](#).

[Buildroot Linux](#) is a "simple, efficient and easy-to-use tool to generate embedded Linux systems through cross-compilation" (<https://buildroot.org/>). In other words, it allows us to compile our own custom version of Linux utilizing only the services/tools we need. For our system to work, our version of Linux required 1) the bare minimum drivers/process such that our hardware architecture worked (CPU, memory, GPIO, HDMI, wireless LAN, and serial USB), 2) a windows manager to display our application and 3) binaries for GDB C and C++ as our application is transcompiled into C++. Once these requirements were configured and compiled, we had our own version of Linux such that our application could run on boot, yet further performance improvements to boot time could still be made. Looking into open-source projects that had utilized Buildroot for fast-booting, we came across [Furkan Tokaç's](#) incredible fastboot project. Shown in [Raspberry Pi 3 Fastboot - Less than 2 seconds](#), Furkan was able to utilize Buildroot to boot a [QT](#) application on a Raspberry Pi 3 in an incredibly fast 2.82 seconds. [His article](#) was integral to getting our to system to fastboot, using and adapting his [Buildroot configurations](#). Our configurations were slightly adapted from Furkan's to fit our requirements outlined above by including support for our wireless LAN and USB serial support as they were omitted from his original build.

These final improvements allowed us to reach a boot time of around 2-4 seconds from initial power to application launch.

3.3.2 Node.js & External Dependencies

Unfortunately, we were not able to achieve sub two second launch time. We believe this is due to our choice of development stack, I.E using Node.js/Electron. Node.js is infamously known for having incredibly large dependency packages, and with Electron essentially rewriting our Node.js code into C++ and machine code, our compiled package size is around 1.5 GB. Again, while this hinders our performance, we believed that limiting Bus Factor and maintaining a high level codebase outweighs this downside. However, we tried our best to mitigate ballooning dependency size by cutting down on some 'luxuries' that external third party packages offer.

```
"dependencies": {
  "bootstrap": "^4.5.3",
  "electron-squirrel-startup": "^1.0.0",
  "serialport": "^9.0.2"
},
```

Figure 9: FSAE Dashboard Dependencies

Shown above in [Figure 9](#) is our `package.json` file listing our installed dependencies¹. [Twitter's Bootstrap](#) supplies our CSS and is included mainly for its' ease of adapting to various screen sizes. Part of our development pedagogy was

¹Not including the dependencies included by the dependencies themselves

to plan ahead for project scaling, thus bootstrap allows us to develop for a single screen size yet adapt to any size we may choose without accounting for every possible size, deeming it a necessity. The last two packages are the building blocks to our project: [Serialport](#) is a Node.js library that allows us to setup event handlers and communicate with the on board serial ports on the Raspberry Pi as explained in the Application Architecture section, and [Electron Forge](#) (listed as [electron-squirrel-startup](#)), is the Electron framework itself, allowing us to compile our application into a desktop executable.

We are able to only use three dependencies by using a large portion of the features built in to modern [JavaScript](#) ([ECMAScript 2015](#)) and [HTML 5](#), most notably the use of [HTML Canvas](#). HTML 5's Canvas was used in favor of data visualization libraries such as [D3.js](#) due to the simplicity of our visualization and the need to cut down on overall package size. Which in turn, allows us to save computing power and decrease boot time,

4 Critical Information

In a racing situation, certain sensor information is more critical than others. For example, engine RPM tells the driver when they should shift gears to get the most power out of the engine. The RPM readout is more critical for a driver than something like the injector duty cycle, and as such is prioritised on the dashboard display. Additionally, too much information on the dashboard would distract the driver. After consulting the drivers and the ergonomics lead, we decided that the dashboard would display engine RPM and the current gear, as those are the most important pieces of information for a driver to have. The RPM would also change color, red indicating the driver should shift into a higher gear, and blue indicating a shift to a lower gear.

5 Complications

Due to Covid-19, our team had limited access to testing on the actual car, and had to do the design and testing off the car. We were not able to drive the car with the dashboard attached, and all the CAN communications took place on a static test bench. Without driver feedback there is no way to know if the data displayed is actually helpful in a race scenario.

6 Future Works

This year was the first in which our team attempted to create a digital dashboard for our car. Therefore, this is the very beginning of our project and we plan to continue development to upkeep the system. For the upcoming year we plan on mounting the dashboard onto the car, and having testing time on the real vehicle. Depending on driver feedback, the display size and brightness might need to be changed, as well as the graphics and data displayed. Our plan is to make a system, such that, the dashboard can be easily changed by the drivers to show the information they see fit. Our current solution requires a whole recompilation of the application to integrate new changes. As more sensors are implemented onto the car, we hope to add driver controls to switch between different pages of data on the dashboard. Our use of a Raspberry Pi 4 computer allows us to plan on easily adding these upgrades without the need of a complete redesign. We are barely using less than half of its available memory, and we will integrate more features in the future as we see fit.